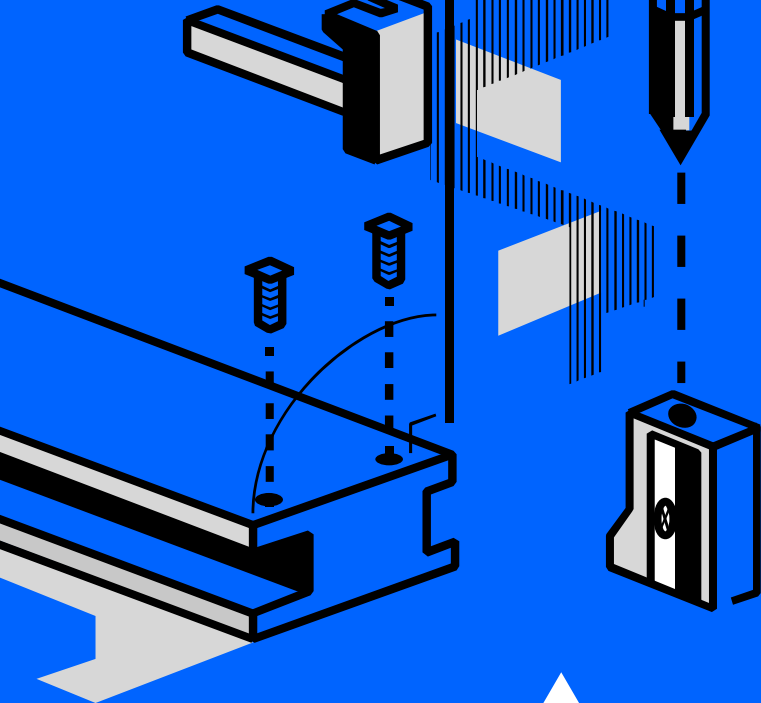


Full Stack- server

יסודות Node.js והקמת פרויקט

המכללה
הטכנולוגית
באר שבע

בשיתוף WORLD ORT
קדימה מדע



Server 101

•מה נלמד היום?

- מה זה Node.js? (הקונספט)
- ארכיטקטורת Event Loop ו-Non-blocking I/O
- יצירת פרויקט Express (באמצעות `express-generator`)
- הגדרת ES Modules
- התקנת חבילות חיוניות (`dotenv`, `cors`, `nodemon`)
- מבנה קובץ `app.js` בסיס

מה זה Node.js

סביבת ריצה (Runtime) של **JavaScript** בצד השרת.
מבוסס על מנוע V8 של כרום (אותו מנוע שרץ בדפדפן).
הרעיון: "**JavaScript** בכל מקום" - אותה שפה ל-**Frontend** ול-**Backend**.

מודל הפעולה: Event Loop

Node.js פועל על תהליך (Thread) יחיד.

Non-blocking I/O: פעולות "כבדות" (כמו קריאה ממסד נתונים, קריאת קובץ) לא "תוקעות" את התוכנית. הן נשלחות לעבודה ברקע.

Event Loop: "לולאה" שבודקת כל הזמן אם פעולות רקע הסתיימו, ומפעילה את ה-Callback שלהן כשהן מוכנות.

התוצאה: יכולת לטפל באלפי חיבורים במקביל ביעילות גבוהה.

הקמת פרויקט

שלב 2: התקנת Express

```
npm install express
```

אנחנו נבנה את הפרויקט מהיסוד כדי להבין כל שלב.

שלב 1: יצירת תיקייה ואתחול הפרויקט

```
mkdir my-express-server  
cd my-express-server  
npm init -y
```

`npm init -y`: יוצר קובץ `package.json` בסיסי בתיקייה (הדגל `-y` מאשר אוטומטית את כל השאלות).

עדכון ל-ES Modules

לאחר ש-`npm init` יצר את `package.json`, ניגש לערוך אותו.
נוסיף שורה אחת כדי להגדיר שהפרויקט ישתמש ב-`import` ו-`export` מודרניים:

```
{
  "name": "my-express-server",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "type": "module",  <-- להוסיף את השורה הזאת
  "scripts": { ... }
}
```

למה? בלי הגדרה זו, היינו צריכים להשתמש בתחביר הישן (`require()`)

התקנת חבילות נדרשות נוספות

אלו חבילות שתמיד נרצה בפרויקט Express מודרני:

```
npm install dotenv cors
```

```
npm install nodemon --save-dev
```

dotenv: לטעינת משתני סביבה (כמו מפתחות סודיים) מקובץ **.env**.
cors: מאפשר לשרת שלנו לקבל בקשות מדומיינים אחרים (למשל, אפליקציית React שרצה בפורט 3000).
nodemon: כלי פיתוח שמפעיל מחדש את השרת אוטומטית בכל שינוי קוד (מותקן כ-**devDependency**).

package.json והוספת סקריפטים

מהו הקובץ package.json?

- זהו "תעודת זהות" של פרויקט Node.js. הוא נוצר אוטומטית כשהרצנו `npm init`.
- תפקידיו המרכזיים:

1. **ניהול תלויות (Dependencies):** הוא מחזיק רשימה מדויקת של כל החבילות שהתקנו (כמו `express`, `cors`, `nodemon`) והגרסאות שלהן.
2. **ניהול סקריפטים (Scripts):** הוא מאפשר לנו להגדיר "קיצורי דרך" (פקודות) להרצת משימות נפוצות בפרויקט.

הוספת סקריפטים לפיתוח:

- נוסיף לקובץ `package.json` את האובייקט `scripts` כדי להריץ את השרת בקלות (בהנח
- ה שהקובץ הראשי שלנו הוא `app.js`):

```
"scripts": {  
  "start": "node app.js",  
  "dev": "nodemon app.js"  
},
```

כך נריץ את השרת:

- במקום לכתוב `node app.js`, נריץ:
 - `npm start` (יריץ את השרת לפרודקשן).
- במקום לכתוב `nodemon app.js`, נריץ:
 - `npm run dev` (יריץ את השרת לפיתוח עם ריסטרט אוטומטי).

הקובץ .env (משתני סביבה)



מהו הקובץ .env?

- הקובץ .env (קיצור של Environment, סביבה) הוא קובץ טקסט פשוט שבו אנו מאחסנים את כל הגדרות התצורה (Configuration) והמידע הרגיש של הפרויקט.
- הוא מאפשר לנו להפריד בין הקוד שלנו (שנכנס ל-Git) לבין ההגדרות (שנשארות מקומיות).

למה הוא כל כך חשוב?

1. **אבטחה:** הוא המקום לשמור **מידע רגיש** שאסור שייכנס לקוד המקור.
 - **דוגמאות:** ססמאות למסד נתונים, מפתחות API של שירותים חיצוניים (כמו שליחת מיילים), "סוד" לחתימת טוקנים (JWT).
2. **גמישות:** מאפשר להריץ את אותו קוד בדיוק בסביבות שונות (מחשב פיתוח, שרת בדיקות, פרודקשן) עם הגדרות שונות, פשוט על ידי החלפת הקובץ .env.

איך זה עובד?

- ספריית **dotenv** (שהתקנו מקודם) נטענת בתחילת ריצת השרת (בקובץ `app.js`).
- היא קוראת את הקובץ .env, ומזריקה את כל המשתנים שבו לתוך אובייקט גלובלי של Node שנקרא `process.env`.
- כך, הקוד שלנו יכול לגשת לערכים האלה (למשל `process.env.PORT`).

הקובץ .env (משתני סביבה)



דוגמה ליצירת הקובץ:

- ניצור קובץ בשם .env בתיקייה הראשית:

```
PORT=5000
MONGO_URI=mongodb://myuser:mypassword@localhost:27017/myDb
JWT_SECRET=ThisIsMySuperSecretKeyForTokens123!
```

אזהרת אבטחה קריטית:

- חובה להוסיף את הקובץ .env לקובץ ה-.gitignore של הפרויקט.
- למה? אם הקובץ יעלה בטעות ל-GitHub, כל הסמאות, המפתחות וסודות ה-API שלכם יהיו חשופים לעיני כולם!

מבנה דף ראשי (app.js) - חלק א' (בנייה)

ניצור קובץ חדש: **app.js**
1. ייבוא חבילות:

```
import express from 'express';    //  
import dotenv from 'dotenv';      //  
import cors from 'cors';          //
```

2. טעינת משתנים מ- **env**: (חייב להיות לפני
שמתמשים במשתנים)

```
dotenv.config();
```

3. יצירת אפליקציית Express:

```
const app = express();
```

מבנה דף ראשי (app.js) - חלק ב' (Middleware)

4. שימוש ב-Middleware בסיסי: (נרחיב על זה בשיעור הבא)

```
app.use(cors()); // מאפשר גישה מכל דומיין  
app.use(express.json()); // מאפשר לשרת לקרוא body-מה JSON
```

6. יצירת ראוט (Route) בסיסי:

```
app.get('/', (req, res) => { //  
  res.send('Express! ברוך הבא לשרת'); //  
});
```

5. הגדרת פורט:

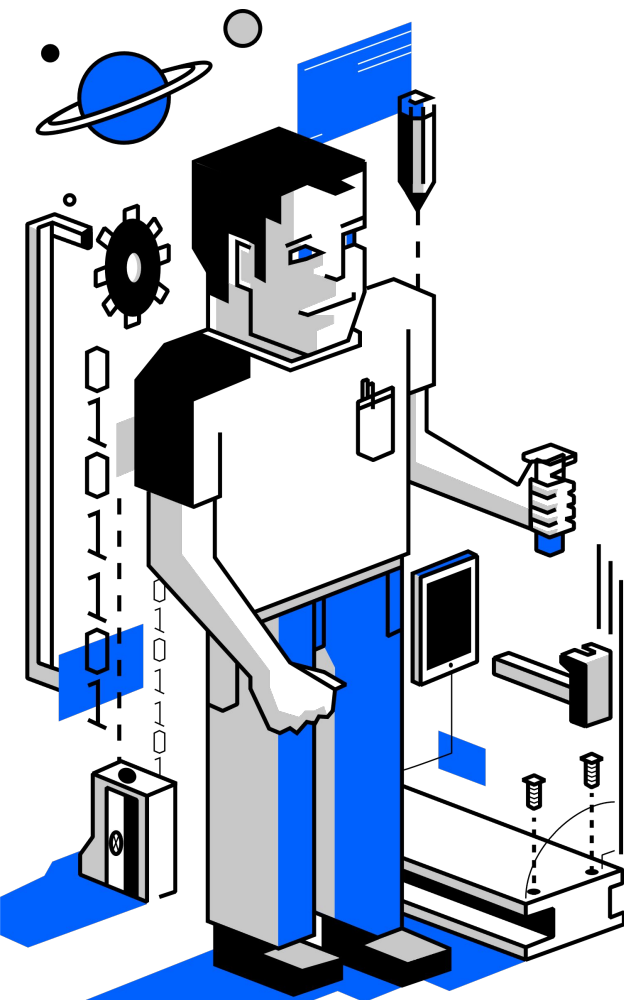
```
const PORT = process.env.PORT || 5000;
```

7. הפעלת השרת (האזנה):

```
app.listen(PORT, () => { //  
  console.log(`Server is running on  
  http://localhost:${PORT}`); //  
});
```

סיכום ושלב הבא

- בנינו שרת Express מאפס!
- איתחלנו פרויקט (**npm init**), התקנו את **express** וחבילות נוספות, והגדרנו ES Modules.
- יצרנו קובץ **app.js** שמטפל ב-**json**, **cors**, ומגיב לראוט בסיסי.
- השרת שלנו רץ עם **nodemon** ומוכן לפיתוח.
- **במצגת הבאה:** נצלול לתוך המושג **Middleware** ונלמד איך לבנות **Routing** (ניתוב) בצורה מסודרת ומודולרית.
-



שאלות